

Protokoły sieciowe

Transfer plików

Zajęcia nr 5



Spis treści

PRZYGOTOWANIE ŚRODOWISKA

Uruchamianie maszyn	5
Połączenie OpenVPN	7
Teoria	9

ZADANIA

Protokoły TCP	17
Protokoły UDP	18
5.1 Netcat File Download (TCPc)	21
5.2 Netcat File Upload (TCPc)	23
5.3 Netcat File Download (UDPc)	24
5.4 Netcat File Upload (UDPc)	25
5.5 Netcat File Download (TCPl)	27
5.6 Netcat File Upload (TCPl)	29
5.7 Netcat File Download (UDPl)	31
5.8 Netcat File Upload (UDPl)	33
Protokoły HTTP i HTTPS	35
5.9 Serwer HTTP (Python)	38
5.10 Serwer HTTP (Apache)	41
5.11 Serwer HTTPS (Apache)	44
Protokół FTP	47
5.12 Serwer FTP (Python)	49
Protokół SMB	52
5.13 Serwer SMB (Python)	54
Protokół SCP	57
5.14 Serwer SCP (SSH)	59

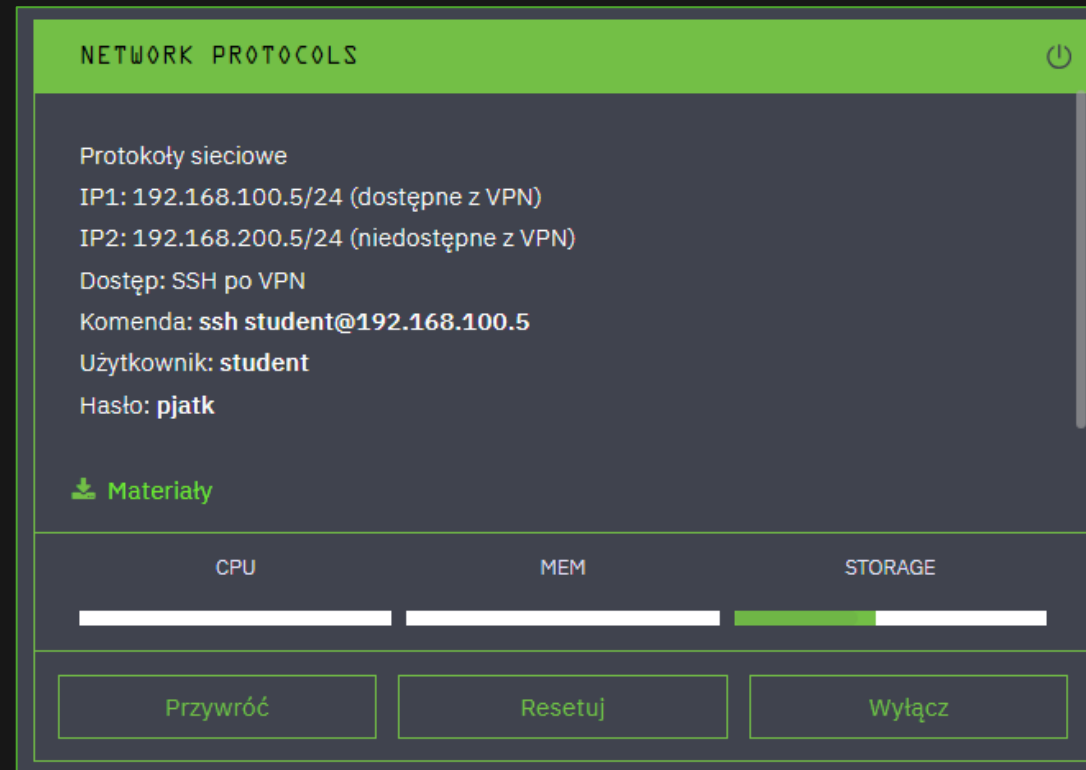
Wstęp

Student nauczy się pobierać i wysyłać pliki przy użyciu różnych protokołów sieciowych. Będą to protokoły warstwy czwartej, takie jak TCP i UDP oraz protokoły warstw wyższych, takie jak: HTTP, HTTPS, FTP, SMB oraz SCP.

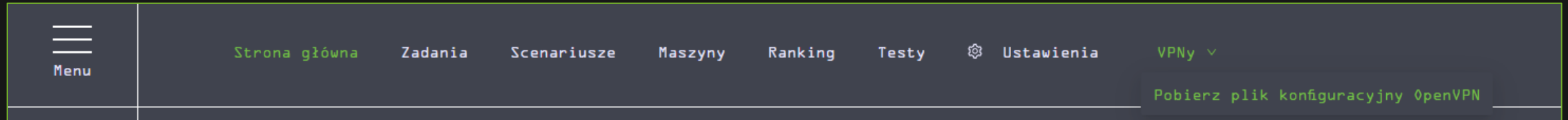
Przygotowanie środowiska

Uruchamianie maszyn

Przed rozpoczęciem upewnij się, że maszyna *NETWORK PROTOCOLS* jest włączona. Jeżeli coś się zepsuje, w każdej chwili możesz ją **przywrócić** do stanu początkowego.



Połączenie OpenVPN



Połączenie OpenVPN jest niezbędne do rozwiązywania zadań. Aby połączyć się z siecią VPN, należy pobrać plik konfiguracyjny OpenVPN z rozwijanego menu „**VPNy**” na stronie głównej. Następnie należy wykonać komendę:

```
sudo openvpn /sciezka/do/pliku.ovpn
```

Legenda

`cat /etc/passwd` - komendę należy wykonać na maszynie Kali

`cat /etc/passwd` - komendę należy wykonać na maszynie Ćwiczenia

`cat /etc/passwd` - komendę należy wykonać na maszynie Pentest

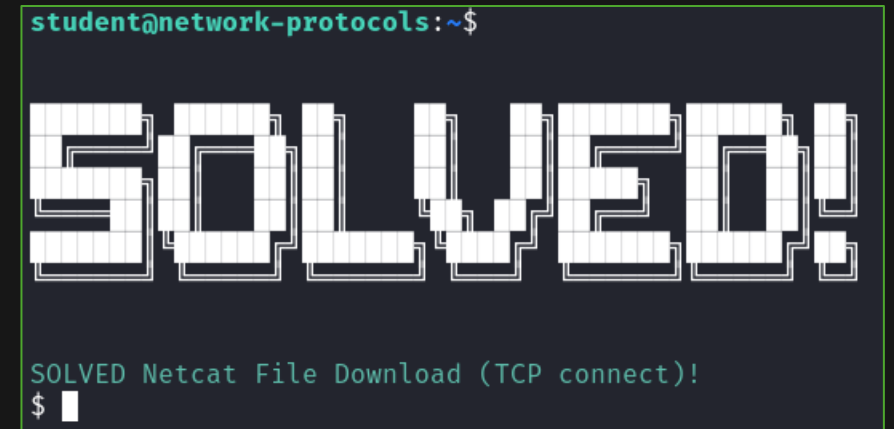
Śledzenie wyników w konsoli

Krok opcjonalny - w celu śledzenia aktualnych postępów w konsoli, istnieje możliwość połączenia się za maszyną z zadaniami, która w czasie rzeczywistym będzie wyświetlać wszystkie wykonane zadania. Uruchamiamy to w osobnej karcie terminala i zostawiamy ją:

```
ssh student@192.168.100.5 #hasło: pjabat
```

Przykładowy ekran po rozwiązaniu zadania:

Zadania rozwiązujemy nadal z Kali!!!



Wyniki można śledzić również na platformie WWW z poziomu mapy na stronie głównej, zakładki „Zadania” lub Rankingu.

Teoria

Rozwój sieci komputerowych

Historia protokołów sieciowych sięga początków informatyki i pierwszych eksperymentów nad komunikacją między komputerami. W latach 60. XX wieku, wraz z rozwojem technologii wojskowej i naukowej, zaczęto tworzyć pierwsze sieci komputerowe, które miały umożliwić niezawodną wymianę danych. Jednym z najważniejszych projektów tamtego okresu była sieć ARPANET, stworzona przez amerykańską agencję DARPA. Był to prekursor dzisiejszego Internetu i jednocześnie środowisko testowe dla pierwszych protokołów komunikacyjnych.

Wraz z rozwojem komputerów osobistych i wzrostem liczby użytkowników sieci pojawiło się zapotrzebowanie na ustandaryzowanie zasad przesyłania danych. W latach 70. opracowano model OSI (Open Systems Interconnection), który zdefiniował warstwy komunikacji sieciowej. Model ten nie stał się powszechnym standardem, ale zainspirował rozwój stosu protokołów TCP/IP, który do dziś dominuje w sieciach komputerowych.

Dlaczego istnieje wiele różnych protokołów?

Zapotrzebowanie na różne protokoły wynika z wielości zastosowań i wymagań technicznych w komunikacji sieciowej. Nie każdy rodzaj transmisji danych wymaga tych samych mechanizmów, dlatego powstały wyspecjalizowane protokoły, dostosowane do różnych zadań. Niektóre zapewniają wysoką niezawodność i kontrolę nad danymi (np. TCP), inne kładą nacisk na szybkość (np. UDP). Istnieją też protokoły dedykowane do udostępniania plików (np. SMB, FTP), przesyłania poczty elektronicznej (SMTP, IMAP, POP3) czy zabezpieczania komunikacji (SSL/TLS).

Alternatywne metody udostępniania plików

Chociaż protokoły takie jak FTP, SMB czy NFS są szeroko stosowane do udostępniania plików w sieciach lokalnych i globalnych, istnieją również alternatywne podejścia. Jednym z nich są systemy **Peer-to-Peer (P2P)**, które pozwalają użytkownikom wymieniać pliki bez konieczności korzystania z centralnego serwera. Przykłady takich rozwiązań to **BitTorrent** czy **eDonkey**, które umożliwiają dystrybucję dużych plików między wieloma użytkownikami bez nadmiernego obciążenia pojedynczego serwera. Wykorzystanie technologii P2P budzi kontrowersje, ponieważ często służy do udostępniania treści chronionych prawem autorskim, takich jak filmy, muzyka czy oprogramowanie. Jednak nie jest to jedyne zastosowanie tej technologii - P2P znajduje również legalne i innowacyjne zastosowania, np. w dystrybucji aktualizacji systemowych, przesyłaniu danych w zdecentralizowanych systemach przechowywania oraz wspieraniu działań open-source.

Alternatywne metody udostępniania plików

Innym podejściem jest model **Peer-to-Many (P2M)**, w którym użytkownicy udostępniają pliki poprzez serwisy hostingowe lub e-maile. Ta metoda często wykorzystywana jest do przesyłania dużych plików bez konieczności konfiguracji dedykowanego serwera.

Zadania

Uwaga!

Wszystkie zadania z tego dokumentu wykonujemy na komputerze podłączonym do sieci OpenVPN - może to być dedykowany Kali Linux lub własny komputer.

Protokoły TCP i UDP

Protokół TCP

Protokół TCP (**Transmission Control Protocol**) jest jednym z głównych protokołów w zestawie protokołów internetowych, używanym do zapewnienia niezawodnej, **uporządkowanej** transmisji danych między komputerami. TCP ustanawia połączenie między dwoma punktami końcowymi (hostami) przed rozpoczęciem przesyłania danych, co jest realizowane za pomocą procesu trójfazowego uzgadniania (**three-way handshake**). Każda wysłana paczka danych jest numerowana, a **odbiorca potwierdza ich odbiór**, co pozwala na wykrycie i **retransmisję** zagubionych lub uszkodzonych paczek. TCP kontroluje przepływ danych, aby uniknąć przeciążenia sieci oraz zapewnia, że dane są odbierane w takiej samej kolejności, w jakiej zostały wysłane.

Protokół TCP jest protokołem **warstwy czwartej** i zapewnia stabilność połączenia. Protokoły wyższych warstw, tj. HTTP, HTTPS, FTP, SMB, SCP wykorzystują niezawodność protokołu TCP do stabilnego działania i niestratnego przesyłania informacji.

Protokół UDP

Protokół UDP (**User Datagram Protocol**) jest jednym z głównych protokołów w zestawie protokołów internetowych, używanym do przesyłania danych w sposób szybki, ale **bez gwarancji ich niezawodności**. W przeciwieństwie do TCP, UDP nie ustanawia połączenia przed rozpoczęciem transmisji danych i **nie zapewnia mechanizmów potwierdzania odbioru ani retransmisji** zagubionych paczek. Każda wysyłana paczka (datagram) jest niezależna, co oznacza, że mogą one dotrzeć do celu w różnej kolejności lub wcale. UDP jest stosowany tam, gdzie **szybkość transmisji jest ważniejsza niż jej niezawodność**, np. w transmisji strumieni wideo lub gier online. Przykładem działania UDP jest transmisja meczu piłkarskiego na żywo: dane wideo są wysyłane w datagramach, które nie muszą być potwierdzane, co zapewnia płynność transmisji, nawet jeśli niektóre fragmenty obrazu mogą zostać utracone.

Zadanie 5.1: Netcat File Download (TCP connect)

Celem zadania jest ściągnięcie pliku hostowanego przez maszynę **192.168.100.5** na porcie **10001/tcp**. Plik zostanie wykorzystany w kolejnym zadaniu.

Plik można pobrać za pomocą narzędzia nc:

```
nc 192.168.100.5 10001 > tcpconnect.png
```

alternatywnie socat:

```
socat -u tcp-connect:192.168.100.5:10001 open:tcpconnect.png,creat
```

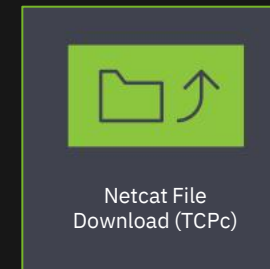
sha1 pliku: 7a3b0cb8d3afa5731f2565f7c6b8d0920c3078c8

Zadanie 5.1 : Netcat File Download (TCP connect)

Zadanie powinno zostać automatycznie zaliczone po pobraniu pliku:

```
(student@HD-Kali)-[~/pjatk]
$ nc 192.168.100.5 10001 > tcpconnect.png

(student@HD-Kali)-[~/pjatk]
$ sha1sum tcpconnect.png
7a3b0cb8d3afa5731f2565f7c6b8d0920c3078c8  tcpconnect.png
```



Zadanie 5.2: Netcat File Upload (TCP connect)

Celem zadania jest wysłanie pliku z poprzedniego zadania na port **20001/tcp** do hosta **192.168.100.5**.

```
cat tcpconnect.png | nc 192.168.100.5 20001
```

alternatywnie socat:

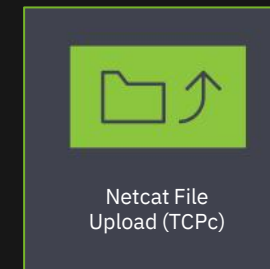
```
socat -u file:tcpconnect.png tcp-listen:20001,reuseaddr
```

Zadanie 5.2: Netcat File Upload (TCP connect)

Zadanie powinno zostać automatycznie zaliczone po wystąpieniu pliku:

```
(student@HD-Kali)-[~/pjak]
$ cat tcpconnect.png | nc 192.168.100.5 20001
```

UPLOADED!



Zadanie 5.3: Netcat File Download (UDP connect)

Celem zadania jest ściągnięcie pliku zahostowanego na porcie **10002/udp** na maszynie **192.168.100.5**. Plik zostanie wykorzystany w kolejnym zadaniu.

sha1: d74422ce9a762669913bee50c171230e9d7d7666

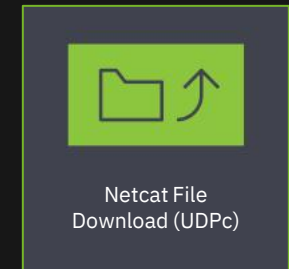
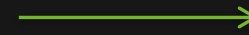
```
echo | nc -u 192.168.100.5 10002 -w 5 > udpconnect.png
```

Zadanie 5.3: Netcat File Download (UDP connect)

Zadanie powinno zostać automatycznie zaliczone po pobraniu pliku:

```
(student@HD-Kali)-[~/pjatk]
$ echo | nc -u 192.168.100.5 10002 -w 5 > udpconnect.png

(student@HD-Kali)-[~/pjatk]
$ sha1sum udpconnect.png
d74422ce9a762669913bee50c171230e9d7d7666  udpconnect.png
```



Zadanie 5.4: Netcat File Upload (UDP connect)

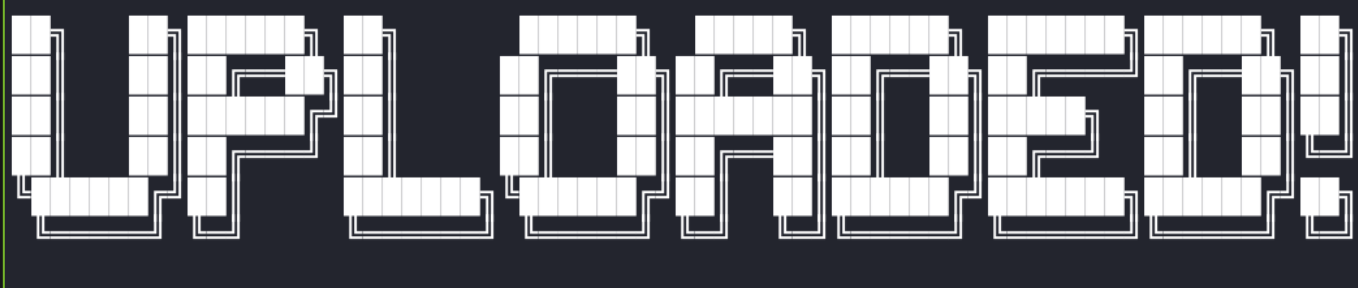
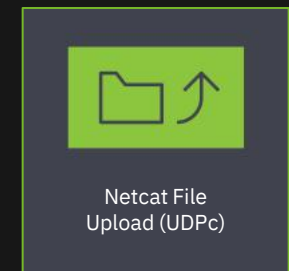
Celem zadania jest wysłanie pliku z poprzedniego zadania na port **20002/udp** do hosta **192.168.100.5**.

```
cat udpconnect.png | nc -u 192.168.100.5 20002 -w 5
```

Zadanie 5.4: Netcat File Upload (UDP connect)

Zadanie powinno zostać automatycznie zaliczone po wysłaniu pliku:

```
(student@HD-Kali)-[~/pjak]
$ cat udpconnect.png | nc -u 192.168.100.5 20002 -w 5
```

Zadanie 5.5: Netcat File Download (TCP listen)

Celem zadania jest ściągnięcie pliku wysyłanego co około 30 sekund na port **10003/tcp** na maszynę (Twoją) podłączoną do sieci VPN. Plik zostanie wykorzystany w kolejnym zadaniu.

sha1: 922acabf8a6ece4103335e2d87e89341d67c1cc5

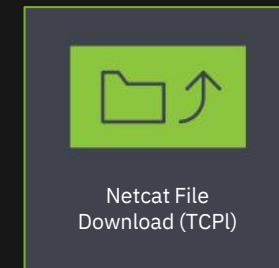
```
nc -lvp 10003 > tcplisten.png
```

Zadanie 5.5: Netcat File Download (TCP listen)

Zadanie powinno zostać automatycznie zaliczone po pobraniu pliku:

```
(student@HD-Kali)-[~/pjak]
$ nc -lvp 10003 > tcplisten.png
listening on [any] 10003 ...
192.168.100.5: inverse host lookup failed: Unknown host
connect to [10.65.0.6] from (UNKNOWN) [192.168.100.5] 43775

(student@HD-Kali)-[~/pjak]
$ sha1sum tcplisten.png
922acabf8a6ece4103335e2d87e89341d67c1cc5  tcplisten.png
```



Zadanie 5.6: Netcat File Upload (TCP listen)

Celem zadania jest zahostowanie pliku z poprzedniego zadania na porcie **20003/tcp** na maszynie (Twojej) podłączonej do sieci VPN.

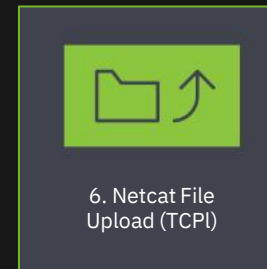
```
cat tcplisten.png | nc -lvp 20003
```

Zadanie 5.6: Netcat File Upload (TCP listen)

Zadanie powinno zostać automatycznie zaliczone po wysłaniu pliku:

```
(student@HD-Kali)-[~/pjatk]
$ cat tcplisten.png | nc -lvp 20003
listening on [any] 20003 ...
192.168.100.5: inverse host lookup failed: Unknown host
connect to [10.65.0.6] from (UNKNOWN) [192.168.100.5] 41233
```

UPLOADED!



Zadanie 5.7: Netcat File Download (UDP listen)

Celem zadania jest ściągnięcie pliku wysyłanego co około 30 sekund na port **10004/udp** na maszynę (Twoją) podłączoną do sieci VPN. Plik zostanie wykorzystany w kolejnym zadaniu.

Po otrzymaniu komunikatu o nawiązaniu połączenia możemy wcisnąć kombinację CTRL-C lub poczekać na timeout (-w 30).

sha1: d18885eaf46a5da871e1d3f69ed79047cf5acc94

Uwaga! Połączenie UDP bywa zawodne. Sprawdź, czy plik został pobrany poprawnie.

```
nc -ludp 10004 -w 30 > udplisten.png
```

Zadanie 5.7: Netcat File Download (UDP listen)

Zadanie powinno zostać automatycznie zaliczone po pobraniu pliku:

```
(student@HD-Kali)-[~/pjak]
$ nc -ludp 10004 -w 30 > udplisten.png
listening on [any] 10004 ...
192.168.100.5: inverse host lookup failed: Unknown host
connect to [10.65.0.6] from (UNKNOWN) [192.168.100.5] 59429
^C

(student@HD-Kali)-[~/pjak]
$ sha1sum udplisten.png
d18885eaf46a5da871e1d3f69ed79047cf5acc94  udplisten.png
```



Zadanie 5.8: Netcat File Upload (UDP listen)

Celem zadania jest zahostowanie pliku z poprzedniego zadania na porcie **20004/udp** na maszynie (Twojej) podłączonej do sieci VPN.

```
cat udplisten.png | nc -ulvp 20004 #CTRL-C po pojawieniu się napisu "UPLOADED"
```

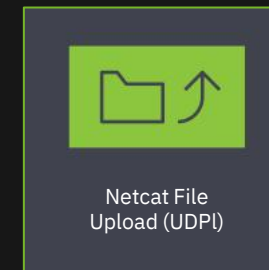
Zadanie 5.8: Netcat File Upload (UDP listen)

Zadanie powinno zostać automatycznie zaliczone po wysłaniu pliku:

```
(student@HD-Kali)-[~/pjatk]
$ cat udplisten.png | nc -u1vp 20004
listening on [any] 20004 ...
192.168.100.5: inverse host lookup failed: Unknown host
connect to [10.65.0.6] from (UNKNOWN) [192.168.100.5] 51434

UPLOADED!
```

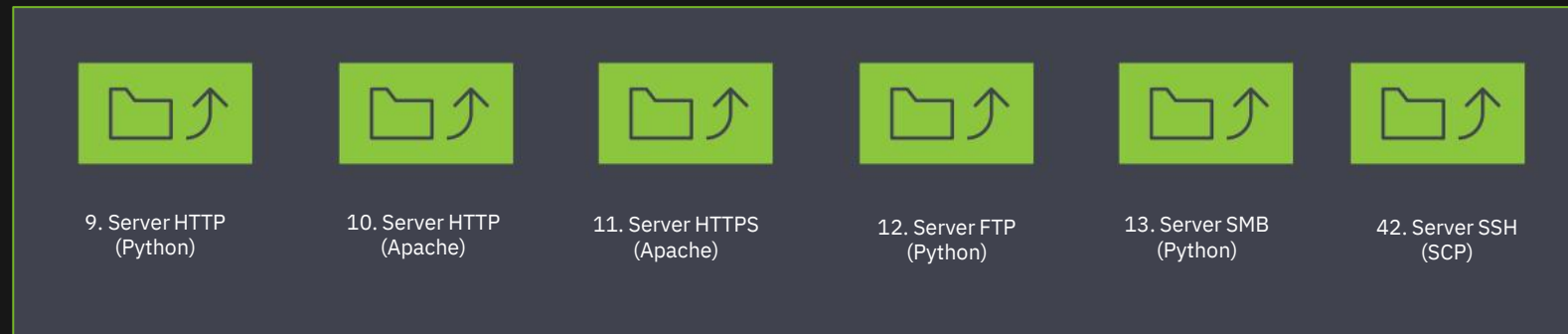
^C



Protokoły HTTP i HTTPS

Automatyczne rozwiązywanie zadań

Wszystkie kolejne zadania powinny zostać automatycznie zaliczone po uruchomieniu konkretnego typu serwera:



Protokoły HTTP i HTTPS

Protokół **HTTP** (Hypertext Transfer Protocol) **umożliwia komunikację między przeglądarką internetową a serwerem**, działając na zasadzie żądań i odpowiedzi: przeglądarka wysyła żądanie, a serwer odpowiada danymi, takimi jak strony internetowe czy obrazy. HTTP jest niezaszyfrowany, co oznacza, że dane mogą być łatwo przechwycone.

HTTPS (Hypertext Transfer Protocol Secure) jest rozszerzeniem HTTP, które **dodaje warstwę szyfrowania** za pomocą **SSL/TLS**, zapewniając bezpieczeństwo przesyłanych danych i chroniąc przed podsłuchiwaniami oraz atakami typu man-in-the-middle.

Zadanie 5.9: Serwer HTTP (Python)

Celem zadania jest uruchomienie serwera HTTP na porcie 80 za pomocą Pythona:

```
python3 -m http.server 80
```

Zadanie 5.9: Serwer HTTP (Python)

Zadanie powinno zostać automatycznie, gdy uruchomiony serwer otrzyma automatycznie wysyłane zapytanie HTTP:

```
(student@HD-Kali)-[~/pjak]  
$ python3 -m http.server 80  
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...  
192.168.100.5 - - [01/Jul/2024 15:25:44] "GET / HTTP/1.1" 200 -
```



Zadanie 5.9: Serwer HTTP (Python)

Efekt działania serwera uruchomionego w katalogu domowym użytkownika po wejściu na <http://localhost/>.

Directory listing for /

- [.bash_logout](#)
- [.bashrc](#)
- [.bashrc.original](#)
- [.cache/](#)
- [.cargo/](#)
- [.config/](#)
- [.dmrc](#)
- [.face](#)
- [.face.icon@](#)
- [.gnupg/](#)
- [.ICEauthority](#)
- [.java/](#)
- [.local/](#)
- [.mozilla/](#)
- [.profile](#)

Zadanie 5.10: Serwer HTTP (Apache)

Celem zadania jest uruchomienie serwera HTTP Apache2 na porcie 80:

```
sudo apt update && sudo apt install apache2 #jeżeli apache2  
nie jest zainstalowany  
sudo systemctl start apache2
```

Zadanie 5.10: Serwer HTTP (Apache)

Zadanie powinno zostać automatycznie zaliczone po uruchomieniu serwera Apache2:

```
(student@HD-Kali)-[~/pjatk]
$ sudo apt update && sudo apt install apache2
Hit:1 http://http.kali.org/kali kali-rolling InRelease
465 packages can be upgraded. Run 'apt list --upgradable' to see them.
apache2 is already the newest version (2.4.59-2).
apache2 set to manually installed.
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 465

(student@HD-Kali)-[~/pjatk]
$ sudo systemctl start apache2
```



Zadanie 5.10: Serwer HTTP (Apache)

Efekt działania serwera po wejściu na <http://localhost/>.



Apache2 Debian Default Page



It works!

This is the default welcome page used to test the correct operation of the Apache2 server after installation on Debian systems. If you can read this page, it means that the Apache HTTP server installed at this site is working properly. You should **replace this file** (located at `/var/www/html/index.html`) before continuing to operate your HTTP server.

If you are a normal user of this web site and don't know what this page is about, this probably means that the site is currently unavailable due to maintenance. If the problem persists, please contact the site's administrator.

Configuration Overview

Debian's Apache2 default configuration is different from the upstream default configuration, and split into several files optimized for interaction with Debian tools. The configuration system is **fully documented in `/usr/share/doc/apache2/README.Debian.gz`**. Refer to this for the full documentation. Documentation for the web server itself can be found by accessing the **manual** if the `apache2-doc` package was installed on this server.

The configuration layout for an Apache2 web server installation on Debian systems is as follows:

```

/etc/apache2/
|-- apache2.conf
|   `-- ports.conf
|-- mods-enabled
|   |-- *.load
|   `-- *.conf
-- conf-enabled

```

Zadanie 5.11: Serwer HTTPS (Apache)

Celem zadania jest uruchomienie serwera HTTPS Apache2 na porcie 443:

```
sudo a2enmod ssl  
sudo a2ensite default-ssl.conf  
sudo systemctl restart apache2
```

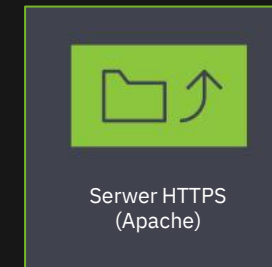
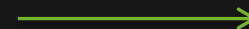
Zadanie 5.11: Serwer HTTPS (Apache)

Zadanie powinno zostać automatycznie zaliczone po uruchomieniu modułu SSL, domyślnej strony https oraz zrestartowaniu serwera Apache2:

```
(student@HD-Kali)-[~/pjak]
$ sudo a2enmod ssl
Considering dependency mime for ssl:
Module mime already enabled
Considering dependency socache_shmcb for ssl:
Module socache_shmcb already enabled
Enabling module ssl.
See /usr/share/doc/apache2/README.Debian.gz on how to configure SSL and create self
-signed certificates.
To activate the new configuration, you need to run:
    systemctl restart apache2

(student@HD-Kali)-[~/pjak]
$ sudo a2ensite default-ssl.conf
Enabling site default-ssl.
To activate the new configuration, you need to run:
    systemctl reload apache2

(student@HD-Kali)-[~/pjak]
$ systemctl restart apache2
```



Zadanie 5.11: Serwer HTTPS (Apache)

Efekt działania serwera po wejściu na <https://localhost/> .
Należy wcześniej zaakceptować niezaufany certyfikat.



Apache2 Debian Default Page



It works!

This is the default welcome page used to test the correct operation of the Apache2 server after installation on Debian systems. If you can read this page, it means that the Apache HTTP server installed at this site is working properly. You should **replace this file** (located at `/var/www/html/index.html`) before continuing to operate your HTTP server.

If you are a normal user of this web site and don't know what this page is about, this probably means that the site is currently unavailable due to maintenance. If the problem persists, please contact the site's administrator.

Configuration Overview

Debian's Apache2 default configuration is different from the upstream default configuration, and split into several files optimized for interaction with Debian tools. The configuration system is **fully documented in `/usr/share/doc/apache2/README.Debian.gz`**. Refer to this for the full documentation. Documentation for the web server itself can be found by accessing the **manual** if the `apache2-doc` package was installed on this server.

The configuration layout for an Apache2 web server installation on Debian systems is as follows:

```

/etc/apache2/
|-- apache2.conf
|   |-- ports.conf
|-- mods-enabled
|   |-- *.load
|   |-- *.conf
|-- conf-enabled

```

Protokół FTP

Protokół FTP

Protokół **FTP** (File Transfer Protocol) jest **używany do przesyłania plików** między komputerami w sieci. Działa na zasadzie połączenia klient-serwer, gdzie klient wysyła żądania do serwera, a serwer odpowiada, umożliwiając przesyłanie plików w obu kierunkach. FTP **nie szyfruje danych**, co oznacza, że przesyłane pliki i dane logowania mogą być łatwo przechwycone przez osoby trzecie. **Istnieje bezpieczna wersja protokołu, FTPS**, która dodaje warstwę szyfrowania za pomocą SSL/TLS, zapewniając bezpieczne przesyłanie plików.

Zadanie 5.12: Serwer FTP (Python)

Celem zadania jest uruchomienie serwera FTP na porcie 21 za pomocą Pythona:

```
sudo apt update && sudo apt install python3-pyftplib  
python3 -m pyftplib -p 21
```

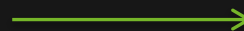
Zadanie 5.12: Serwer FTP (Python)

Zadanie powinno zostać automatycznie zaliczone, gdy uruchomiony serwer otrzyma automatycznie nawiązywane połączenie FTP:

```
(student@student)-[~/pjatk]
$ sudo apt update && sudo apt install python3-pyftplib
Hit:1 http://http.kali.org/kali kali-rolling InRelease
2803 packages can be upgraded. Run 'apt list --upgradable' to see them.
Warning: http://http.kali.org/kali/dists/kali-rolling/InRelease: Key is stored in legacy trusted.gpg keyring (/etc/apt/trusted.gpg), see the DEPRECATION section in apt-key(8) for details.
Installing:
  python3-pyftplib

Summary:
  Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 2803
  Download size: 85.3 kB
  Space needed: 365 kB / 2237 MB available

Get:1 http://mirror.johnnybegood.fr/kali kali-rolling/main amd64 python3-pyftplib all 2.0.1-1 [85.3 kB]
Fetched 85.3 kB in 1s (133 kB/s)
Selecting previously unselected package python3-pyftplib.
(Reading database ... 509672 files and directories currently installed.)
Preparing to unpack .../python3-pyftplib_2.0.1-1_all.deb ...
Unpacking python3-pyftplib (2.0.1-1) ...
Setting up python3-pyftplib (2.0.1-1) ...
Processing triggers for kali-menu (2024.3.1) ...
```



Serwer FTP
(Python)

```
(student@student)-[~/pjatk]
$ python3 -m pyftplib -p 21
[I 2025-06-03 22:25:07] concurrency model: async
[I 2025-06-03 22:25:07] masquerade (NAT) address: None
[I 2025-06-03 22:25:07] passive ports: None
[I 2025-06-03 22:25:07] >>> starting FTP server on 0.0.0.0:21, pid=12418 <<<
[I 2025-06-03 22:25:07] 192.168.100.5:43281-[] FTP session opened (connect)
[I 2025-06-03 22:25:07] 192.168.100.5:43281-[] FTP session closed (disconnect).
[I 2025-06-03 22:26:10] 127.0.0.1:48498-[] FTP session opened (connect)
[I 2025-06-03 22:26:26] 127.0.0.1:48498-[anonymous] USER 'anonymous' logged in.
```

Zadanie 5.12: Serwer FTP (Python)

Efekt działania serwera po połączeniu się z serwerem.
Można zalogować się na użytkownika *anonymous* z dowolnym hasłem.

```
student@HD-Kali)-[~/pjatk]
tnftp localhost
ng [::1]:21 ...
p: Can't connect to `[::1]:21': Connection refused
ng 127.0.0.1:21 ...
ected to localhost.
pyftplib 1.5.9 ready.
(localhost:student): anonymous
Username ok, send password.
word:
Login successful.
te system type is UNIX.
g binary mode to transfer files.
dir
Entering extended passive mode (|||40171|).
Data connection already open. Transfer starting.
rw-r-- 1 student student 50713 tcpconnect.png
rw-r-- 1 student student 46844 tcplisten.png
rw-r-- 1 student student 43141 udpconnect.png
rw-r-- 1 student student 51607 udplisten.png
Transfer complete.
```

Protokół SMB

Protokół SMB

Protokół SMB (Server Message Block) jest używany do udostępniania plików, drukarek i innych zasobów w sieciach lokalnych. Działa na zasadzie połączenia klient-serwer, gdzie klient wysyła żądania dostępu do zasobów, a serwer odpowiada, umożliwiając ich udostępnienie i modyfikację. SMB w pierwotnej wersji nie szyfrował danych, co narażało je na przechwycenie i ataki. Nowoczesne wersje, takie jak SMB 3.0, wprowadzają mechanizmy szyfrowania i poprawiają bezpieczeństwo transmisji danych. Przykładem zastosowania SMB jest udostępnianie folderów i plików w biurze, gdzie pracownicy mogą wspólnie korzystać z tych samych zasobów sieciowych, a nowoczesne wersje SMB zapewniają ochronę tych danych przed nieuprawnionym dostępem.

Zadanie 5.13: Serwer SMB (Python)

Celem zadania jest uruchomienie serwera SMB na porcie 445 za pomocą Pythona:

```
locate smbserver.py #skrypt domyślnie zainstalowany na Kali  
sudo apt install python3-impacket #komenda do instalacji, jeżeli skrypt nie istnieje  
python3 /usr/share/doc/python3-impacket/examples/smbserver.py -ip 0.0.0.0 -smb2support pjatksmbshare /home/student  
# użycie skryptu smbserver.py może różnić się od użytej wersji. Zaleca się wykorzystanie przełącznika --help w razie problemów.
```

Gdzie:

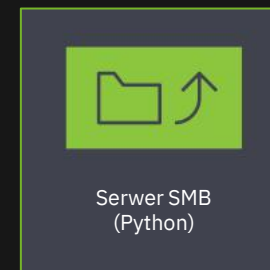
- ip 0.0.0.0 - oznacza nasłuchiwanie na wszystkich interfejsach
- smb2support - oznacza włączenie wsparcia dla nowszej wersji protokołu SMB
- pjatksmbshare - dowolna nazwa udostępnianego zasobu
- /home/student - ścieżka udostępnianego katalogu

Zadanie 5.13: Serwer SMB (Python)

Zadanie powinno zostać automatycznie zaliczone po pobraniu pliku:

```
(student@HD-Kali)-[~/pjatk]
$ locate smbserver.py
/usr/lib/python3/dist-packages/impacket/smbserver.py
/usr/lib/python3/dist-packages/scapy/layers/smbserver.py
/usr/share/doc/python3-impacket/examples/smbserver.py

(student@HD-Kali)-[~/pjatk]
$ python3 /usr/lib/python3/dist-packages/scapy/layers/smbserver.py --iface 0.0.0.0
INFO: -- Scapy 2.5.0+git20240324.2b58b51 SMB Server --
INFO: SSP: NTLM (guest). Serving 1 shares:
INFO: * <SMBShare Scapy [DISKTREE] 'Scapy's SMB server default share' = /home/student/pjatk>
Server SMB_Server started listening on ('0.0.0.0', 445)
| Connection received from ('192.168.100.5', 55224)
> User authenticated as guest!
> Tree Connect on: IPC$
> Opened: srvsvc
<< DceRpc5Bind
>> DceRpc5BindAck srvsvc is on port '\\PIPE\\srvsvc' using NDR 2.0
<< REQUEST: NetrShareEnum_Request
>> RESPONSE: NetrShareEnum_Response
|
```



Zadanie 5.13: Serwer SMB (Python)

Efekt działania serwera po połączeniu się z serwerem.
Można zalogować się bez podawania hasła (-N).

```
(student@HD-Kali)-[~/pjatk]
$ smbclient -N //127.0.0.1/Scapy/
Try "help" to get a list of possible commands.
smb: \> dir
.                D                0
tcpconnect.png   A                50713
tcplisten.png    A                46844
udpconnect.png   A                43141
udplisten.png    A                51607

10485760 blocks of size 4096. 1048576 blocks available
smb: \> █
```

Protokół SCP

Protokół SCP

Protokół SCP (Secure Copy Protocol) służy do bezpiecznego przesyłania plików między komputerami w sieci. Działa na zasadzie połączenia klient-serwer, gdzie pliki są przesyłane za pomocą SSH (Secure Shell), co zapewnia szyfrowanie danych i ochronę przed przechwyceniem. SCP nie wymaga dodatkowej konfiguracji, ponieważ korzysta z istniejącego serwera SSH, co upraszcza jego użycie. Zapewnia integralność i poufność przesyłanych plików, chroniąc je przed nieuprawnionym dostępem i modyfikacją.

Zadanie 5.14: Serwer SSH (SCP)

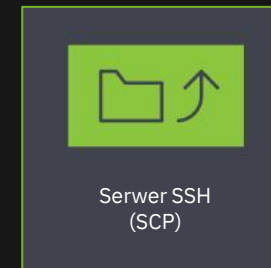
Celem zadania jest uruchomienie serwera SSH (SCP) na porcie 22:

```
sudo apt install openssh-server #jeżeli serwer nie jest zainstalowany  
sudo systemctl start ssh
```

Zadanie 5.14: Serwer SSH (SCP)

Zadanie powinno zostać automatycznie zaliczone po uruchomieniu serwera SSH:

```
(student@HD-Kali)-[~/pjatk]  
$ systemctl start ssh
```



Referencje

<https://www.geeksforgeeks.org/types-of-network-protocols-and-their-uses/>
lista protokołów sieciowych [EN]

<https://www.pluralsight.com/blog/it-ops/networking-basics-tcp-udp-tcpip-osi-models>
różnice między najważniejszymi protokołami [EN]

Koniec ćwiczeń